

APPLIED DEEP LEARNING

Multi-Layer Perceptrons with Keras

Complete Exam-Ready Study Guide

What this document covers

1. The core decision rules — how to tell classification from regression, and which activation, loss, metric, and optimizer to use in each case.
2. How to count trainable weights in an MLP by hand (with fully worked examples).
3. A complete checklist of the bugs that keep showing up in these exercises (shape mismatches, wrong activations, wrong losses, unencoded categorical data, bad batch sizes...).
4. Full, exam-formatted solutions to your Lab 7 exercise and your mock-exam question — written the way you should write them in the exam.
5. A glossary explaining every term (epoch, batch size, one-hot encoding, softmax, sparse categorical crossentropy...) in plain language.

Use Part 1 as your cheat sheet the night before. Use Parts 3–4 to see exactly how full-mark answers are written. Use Part 6 if any term is unclear.

Contents

- 1. Part 1 — The Core Decision Rules (Cheat Sheet)
- 2. Part 2 — Worked Reference Pipeline (the Iris example)
- 3. Part 3 — Fully Solved: Lab 7, Task 1 (Find the Bugs)
- 4. Part 4 — Fully Solved: Mock-Exam Question (Cat / Dog / Hare MLP)
- 5. Part 5 — How to Attack Any "Find the Error" Question
- 6. Part 6 — Glossary of Terms

Part 1 — The Core Decision Rules (Cheat Sheet)

1.1 Is it Classification or Regression?

Look at what y (the target) actually is:

Signal in the data	Task type
y is a class name / integer label (e.g. 0, 1, 2 or "cat", "dog")	Classification
y is one-hot vectors, e.g. [0,1,0]	Classification (multi-class)
y has exactly two possible values (0/1, yes/no)	Binary classification
y is a continuous number (price, temperature, score)	Regression

Rule of thumb: if you could sensibly compute "the average of all the y -values" and that average means something (e.g. average house price), it's regression. If the average would be nonsense (e.g. average of class labels 0,1,2), it's classification.

1.2 Choosing Activation Functions

Hidden layers

- ReLU ("relu") — the default choice for hidden layers in an MLP. Fast, avoids vanishing gradients, almost always safe to use.
- tanh / sigmoid — older alternatives, rarely the default anymore; sigmoid in hidden layers can cause vanishing gradients in deeper nets.

Output layer — this depends entirely on the task

Task	Output units	Output activation	Why
Binary classification	1	sigmoid	Squashes output to (0,1) → interpreted as a probability
Multi-class classification	= number of classes	softmax	Turns raw scores into a probability distribution that sums to 1
Regression	1 (or = number of predicted values)	linear / none	Predicted value can be any real number — ReLU or sigmoid would wrongly restrict the range

Common trap

Using `activation='relu'` on a classification output layer is a classic exam bug. ReLU outputs any non-negative number — it is not a probability, doesn't sum to 1, and pairs badly with a crossentropy loss. If you see `relu` on the final layer of a classifier, that is an error to flag.

1.3 Choosing the Loss Function

Situation	Correct loss
Binary classification (1 output unit, sigmoid)	<code>binary_crossentropy</code>
Multi-class, y given as one-hot vectors (e.g. [0,1,0])	<code>categorical_crossentropy</code>

Multi-class, y given as plain integers (e.g. 0, 1, 2)	<code>sparse_categorical_crossentropy</code>
Regression	<code>mean_squared_error (mse)</code> or <code>mean_absolute_error (mae)</code>

Common trap

`categorical_crossentropy` vs `sparse_categorical_crossentropy` is decided purely by the SHAPE of y, not by "which sounds right": one-hot y → `categorical_crossentropy`; integer y → `sparse_categorical_crossentropy`. Mixing them up throws a shape error at `fit()` time.

1.4 Choosing Metrics

- Classification → 'accuracy' (optionally precision, recall, F1, AUC for imbalanced data).
- Regression → 'mae' (mean absolute error) and/or 'mse' (mean squared error); sometimes RMSE or R^2 .

Common trap

`metrics=['mae']` on a classification model is wrong — MAE measures numeric distance, but class labels/one-hot vectors aren't a numeric scale where "distance" is meaningful. Classification needs 'accuracy' (or similar). Conversely, using 'accuracy' on a pure regression model is meaningless.

1.5 Choosing the Optimizer

- 'adam' — the default, safe choice for almost every MLP exercise. Adapts the learning rate automatically, converges quickly.
- 'sgd' (stochastic gradient descent) — simpler, sometimes used to demonstrate learning-rate/momentum concepts explicitly.
- 'rmsprop' — another adaptive optimizer, common in RNNs but fine for MLPs too.

Unless a question specifically tests optimizer knowledge, 'adam' is essentially always an acceptable / expected choice.

1.6 Counting Trainable Parameters (Weights) — the formula

Formula for one Dense layer

`params = (inputs_to_layer × units) + units` ← the "+ units" term is the bias, one bias per neuron.

Apply this layer by layer, where "inputs_to_layer" is the number of units in the previous layer (or the input dimension for the first layer). Sum across all layers to get the model total.

Worked example

A `Dense(10)` layer following a 4-feature input, then a `Dense(3)` output layer:

- Layer 1: $(4 \times 10) + 10 = 40 + 10 = 50$ parameters
- Layer 2: $(10 \times 3) + 3 = 30 + 3 = 33$ parameters
- Total trainable parameters = $50 + 33 = 83$

This is exactly the 50-and-33 pattern you saw in your lab slides — now you know where both numbers come from and can reproduce this for any architecture.

1.7 Shape & Data-Type Checklist — read every MLP code snippet against this list

- **Input shape** — Does `Input(shape=(n,))` / `input_shape=(n,)` match the actual number of columns in `X`? Count the columns yourself — don't trust the code's comment.
- **Sample count** — Do `X` and `y` have the same number of rows (samples)? `len(X)` must equal `len(y)`.
- **Data types** — Are all features numeric? Strings/categories (e.g. "blue", "red") must be encoded (one-hot or label encoding) before entering a Dense layer.
- **Output units** — Does the final layer's units equal the number of classes (classification) or the number of predicted values (regression)?
- **Output activation** — softmax (multi-class) / sigmoid (binary) / linear (regression) — never relu on a classification output.
- **Loss ↔ y shape** — one-hot `y` → `categorical_crossentropy`; integer `y` → `sparse_categorical_crossentropy`; continuous `y` → `mse/mae`.
- **Metrics** — 'accuracy' for classification, 'mae'/'mse' for regression — never mixed up.
- **batch_size** — `batch_size` should not exceed the number of training samples — with very small datasets (a handful of rows), keep `batch_size` small (e.g. 2-5).

Part 2 — Worked Reference Pipeline (the Iris Example)

This is the "gold standard" pattern from your lab slides — a complete, correct Keras MLP pipeline. Compare every buggy exercise against this shape.

Step 1 — Imports

```
from keras.models import Sequential
from keras.layers import Dense
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelBinarizer
```

Step 2 — Load data, encode labels, split

```
# Load dataset
iris = load_iris()
X = iris.data
y = iris.target

# One-hot encode target labels
encoder = LabelBinarizer()
y = encoder.fit_transform(y)

# Split data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
                                                    random_state=42)
```

Note: y is one-hot encoded here (3 classes → 3-column vectors), which is exactly why the model below must use categorical_crossentropy, not the sparse version.

Step 3 — Define the model

```
# Define MLP model
model = Sequential()
model.add(Dense(10, input_shape=(4,), activation='relu')) # Hidden layer
model.add(Dense(3, activation='softmax'))                  # Output layer
model.summary()
```

- Input dimensionality: 4 (the four iris measurements).
- 2 layers: hidden layer with 10 neurons, output layer with 3 neurons (one per iris species).
- ReLU on the hidden layer, softmax on the output — correct for multi-class classification.
- Parameter count: 50 (layer 1) + 33 (layer 2) = 83 trainable parameters — see §1.6.

Step 4 — Compile, train, evaluate

```
# Compile model
model.compile(optimizer='adam', loss='categorical_crossentropy',
              metrics=['accuracy'])

# Train model
model.fit(X_train, y_train, epochs=50, batch_size=5, verbose=1)

# Evaluate model
```

```
loss, accuracy = model.evaluate(X_test, y_test)

print(f'Test Accuracy: {accuracy}')
```

Why every choice here is correct

adam — safe default optimizer.

categorical_crossentropy — matches one-hot y from LabelBinarizer.

'accuracy' — the right metric for a classification task.

batch_size=5 — small and sensible for a small dataset; never larger than the number of training samples.

Part 3 — Fully Solved: Lab 7, Task 1 (Find the Bugs)

Original (buggy) code from the exercise sheet:

```
import tensorflow as tf
from tensorflow.keras import layers, models
import numpy as np

X = np.array([
    [1.2, 0.7, -3.1, "blue"],
    [0.5, -1.0, 2.2, "red"],
    [-0.2, 4.1, 0.0, "red"],
    [3.3, 1.1, -0.9, "blue"],
    [1.0, 0.0, 1.5, "blue"]
], dtype=object)

y = np.array([0, 2, 1, 0])

model = models.Sequential([
    layers.Dense(16, activation="relu", input_shape=(5,)),
    layers.Dense(8, activation="relu"),
    layers.Dense(3, activation="relu")
])

model.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy",
    metrics=["mae"]
)

model.fit(X, y, epochs=10, batch_size=2)
```

Bug-by-bug analysis

1. Sample-count mismatch between X and y

- **Problem:** X has 5 rows (5 samples), but y has only 4 labels. Keras requires `len(X) == len(y)`; `fit()` will raise a `ValueError` about inconsistent number of samples.
- **Fix:** Add the missing 5th label to y (e.g. `y = np.array([0, 2, 1, 0, 2])`), using whatever the true class of the 5th row is), or remove the extra row from X so both arrays line up.

2. Unencoded categorical feature

- **Problem:** The 4th column is a string ("blue"/"red") mixed with numeric features and stored with `dtype=object`. A `Dense` layer only accepts numeric tensors — it cannot multiply weights by the string "blue".
- **Fix:** Separate the categorical column, encode it numerically (e.g. one-hot encode blue/red with `OneHotEncoder` or `pd.get_dummies`, or label-encode as 0/1), then concatenate it back with the numeric columns before building X.

3. Wrong input_shape

- **Problem:** `input_shape=(5,)` is declared, but each row only has 4 entries (3 numeric + 1 categorical). Even after fixing bug 2, the feature count must be recounted (it may grow if one-hot encoding is used, e.g. 3 numeric + 2 one-hot color columns = 5).
- **Fix:** Set `input_shape` to whatever the true, final number of columns is after preprocessing — count it explicitly, don't guess.

4. Wrong output activation

- **Problem:** The final layer uses `activation="relu"`, but this is a 3-class classification problem ($y \in \{0,1,2\}$). ReLU does not produce a valid probability distribution.
- **Fix:** Change the last layer to `layers.Dense(3, activation="softmax")`.

5. Wrong metric for a classification task

- **Problem:** `metrics=["mae"]` (mean absolute error) is a regression metric; it does not evaluate classification performance meaningfully.
- **Fix:** Use `metrics=["accuracy"]` instead.

What was actually correct (don't 'fix' this!)

Correct as-is

because `y` is given as plain integers (0,1,2), not one-hot vectors. Switching it to plain `categorical_crossentropy` would itself become a bug unless `y` is also one-hot encoded.

Corrected code

```
import tensorflow as tf
from tensorflow.keras import layers, models
import numpy as np
from sklearn.preprocessing import OneHotEncoder

# numeric features
X_num = np.array([
    [1.2, 0.7, -3.1],
    [0.5, -1.0, 2.2],
    [-0.2, 4.1, 0.0],
    [3.3, 1.1, -0.9],
    [1.0, 0.0, 1.5],
])
# categorical feature, encoded
colors = np.array(["blue", "red", "red", "blue", "blue"])
encoder = OneHotEncoder(sparse_output=False)
colors_enc = encoder.fit_transform(colors)

X = np.hstack([X_num, colors_enc]) # final feature matrix
y = np.array([0, 2, 1, 0, 2]) # one label per sample (bug 1 fixed)

model = models.Sequential([
    layers.Dense(16, activation="relu", input_shape=(X.shape[1],)), # bug 3 fixed
    layers.Dense(8, activation="relu"),
    layers.Dense(3, activation="softmax") # bug 4 fixed
])

model.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy", # correct, left unchanged
    metrics=["accuracy"] # bug 5 fixed
)

model.fit(X, y, epochs=10, batch_size=2)
```

Part 4 — Fully Solved: Mock-Exam Question

(Cat / Dog / Hare classification MLP — subquestions i–v, 12 points total)

Exam code:

```
import numpy as np
import tensorflow as tf

X = np.array([
    [1.0, 2.0, 0.5, 0.3], # -> Class: Cat
    [3.0, 1.0, 0.2, 0.1], # -> Class: Dog
    [0.1, 0.4, 2.5, 2.0], # -> Class: Hare
    [2.0, 1.5, 0.4, 0.3], # -> Class: Dog
    [0.2, 0.3, 2.0, 1.8], # -> Class: Hare
])

y = np.array([
    [1, 0, 0], # Cat
    [0, 1, 0], # Dog
    [0, 0, 1], # Hare
    [0, 1, 0], # Dog
    [0, 0, 1], # Hare
])

model = tf.keras.Sequential([
    tf.keras.layers.Input(shape=(8,)),
    tf.keras.layers.Dense(units=4, activation='relu'),
    tf.keras.layers.Dense(units=2, activation='relu'),
    tf.keras.layers.Dense(units=1, activation='relu')
])

model.compile(optimizer='adam', loss='categorical_crossentropy',
metrics=['accuracy'])

model.fit(X, y, epochs=50, batch_size=16)
```

i. What do the variables X and y mean? (2 P)

- **X** is the input feature matrix: 5 samples (animals), each described by 4 numeric measurements/features.
- **y** is the target/label matrix: the ground-truth class for each sample, given as a one-hot encoded vector over 3 classes (Cat, Dog, Hare) — e.g. [1,0,0] means "Cat".

ii. Why does the combination of X and y cause a training error? (2 P)

The Input layer is declared as `Input(shape=(8,))`, i.e. the model expects 8 features per sample. But X actually has only 4 columns per row (shape (5, 4)). This mismatch between the declared input dimension and the real data shape raises a shape-incompatibility error as soon as `fit()` tries to pass X through the first layer.

Correction

Change `tf.keras.layers.Input(shape=(8,))` to `tf.keras.layers.Input(shape=(4,))` so it matches the 4 features actually present in X.

iii. How many trainable weights does the MLP have (with bias neurons)? (2 P)

Using the corrected input dimension of 4 (from part ii) with the layer sizes exactly as declared in the code (Dense(4) → Dense(2) → Dense(1)), and the formula $\text{params} = \text{inputs} \times \text{units} + \text{units}$:

Layer	Calculation	Params
Input (4) → Dense(4)	$(4 \times 4) + 4$	20
Dense(4) → Dense(2)	$(4 \times 2) + 2$	10
Dense(2) → Dense(1)	$(2 \times 1) + 1$	3

Total trainable weights = 20 + 10 + 3 = 33

(If you additionally apply the fix from part iv — output units changed from 1 to 3 — the last layer becomes $(2 \times 3) + 3 = 9$, giving a new total of $20 + 10 + 9 = 39$. State your assumption clearly in the exam: compute it either strictly from the code as given, or note the recalculated total after your correction — examiners generally award marks for showing the correct method.)

iv. Errors in the model definition (lines 19–24) (4 P)

Error A — Wrong input shape

- **Problem:** Input(shape=(8,)) does not match the 4 features in X (same issue as part ii, located here in the model block).
- **Fix:** Change to Input(shape=(4,)).

Error B — Wrong number of output units

- **Problem:** The final Dense layer has units=1, but this is a 3-class problem (Cat/Dog/Hare) and y has 3 columns. A single output neuron cannot represent 3 classes.
- **Fix:** Change to units=3.

Error C — Wrong output activation

- **Problem:** The final layer uses activation='relu'. For multi-class classification with categorical_crossentropy, the output must be a probability distribution over the classes (non-negative values that sum to 1) — ReLU does not guarantee that.
- **Fix:** Change to activation='softmax'.

Corrected model definition:

```
model = tf.keras.Sequential([
    tf.keras.layers.Input(shape=(4,)),
    tf.keras.layers.Dense(units=4, activation='relu'),
    tf.keras.layers.Dense(units=2, activation='relu'),
    tf.keras.layers.Dense(units=3, activation='softmax')
])
```

v. Why does line 28 cause an error during execution? (2 P)

Line 28 is: `model.fit(X, y, epochs=50, batch_size=16)`. There are only 5 training samples in X, but `batch_size=16` asks Keras to form batches of 16 samples. Since 16 is greater than the total number of available samples, Keras cannot build a valid batch and raises an error (e.g. "Expect x to be a non-empty array or dataset" / a shape-related `ValueError`) instead of silently using a smaller batch.

Correction

Reduce batch_size to a value that is $\leq$ the number of training samples, e.g. batch_size=2 or batch_size=5.

Fully corrected code (all parts combined)

```
import numpy as np
import tensorflow as tf

X = np.array([
    [1.0, 2.0, 0.5, 0.3], # Cat
    [3.0, 1.0, 0.2, 0.1], # Dog
    [0.1, 0.4, 2.5, 2.0], # Hare
    [2.0, 1.5, 0.4, 0.3], # Dog
    [0.2, 0.3, 2.0, 1.8], # Hare
])

y = np.array([
    [1, 0, 0], # Cat
    [0, 1, 0], # Dog
    [0, 0, 1], # Hare
    [0, 1, 0], # Dog
    [0, 0, 1], # Hare
])

model = tf.keras.Sequential([
    tf.keras.layers.Input(shape=(4,)), # fixed (ii / iv-A)
    tf.keras.layers.Dense(units=4, activation='relu'),
    tf.keras.layers.Dense(units=2, activation='relu'),
    tf.keras.layers.Dense(units=3, activation='softmax') # fixed (iv-B, iv-
C)
])

model.compile(optimizer='adam', loss='categorical_crossentropy',
metrics=['accuracy'])

model.fit(X, y, epochs=50, batch_size=2) # fixed (v)
```

Part 5 — How to Attack Any "Find the Error" Question

Any exam or lab question that hands you Keras code and asks "what's wrong" is testing the same handful of ideas, over and over, just recombined. Run through this fixed sequence every time — in order — and you will catch almost everything:

1. **Identify the task type.** Look at `y`. Integers/one-hot with a small number of distinct values → classification. Continuous numbers → regression. This determines the correct activation, loss, and metric for the rest of your checks.
2. **Count features in X.** Manually count the columns of `X` (or rows if it's transposed). Compare against `Input(shape=(...)) / input_shape=(...)`. Mismatch = error.
3. **Count samples.** Check `len(X) == len(y)`. A silent off-by-one here is a favourite exam trap.
4. **Check for non-numeric data.** Any strings/categories in `X` must be encoded before touching a Dense layer.
5. **Check the output layer.** units must equal number of classes (classification) or number of targets (regression). activation must be softmax/sigmoid/linear as appropriate — never relu for a classification output.
6. **Check the loss vs. y's shape.** one-hot → `categorical_crossentropy`, integer labels → `sparse_categorical_crossentropy`, continuous → `mse/mae`.
7. **Check the metric.** 'accuracy' for classification, 'mae'/'mse' for regression.
8. **Check fit() arguments.** `batch_size` should not exceed the number of samples; `epochs` and `verbose` don't usually cause errors but sanity-check them too.

Sentence templates you can reuse in the exam

- "This is a [classification/regression] problem because `y` is [one-hot / integer-labelled / continuous]."
- "The declared input shape (`n`) does not match the actual number of features in `X` (`m`), which causes a shape-incompatibility error when `fit()` is called."
- "Because this is multi-class classification, the output layer should use softmax activation with units equal to the number of classes, not [wrong activation/units]."
- "Since `y` is [one-hot / integer], the loss should be [categorical_crossentropy / sparse_categorical_crossentropy], not [wrong loss]."
- "`metrics=['mae']` is a regression metric and is not meaningful for a classification task; it should be replaced with 'accuracy'."
- "`batch_size` (`n`) exceeds the number of training samples (`m`), so Keras cannot form a full batch and raises an error; `batch_size` should be reduced to $\leq m$."

Part 6 — Glossary of Terms

- **MLP (Multi-Layer Perceptron)** — A neural network made of stacked fully-connected (Dense) layers: an input layer, one or more hidden layers, and an output layer.
- **Dense layer** — A fully-connected layer — every input neuron is connected to every output neuron of that layer, each connection having its own weight, plus one bias per output neuron.
- **Weight / parameter** — A learnable number the network adjusts during training. Counted per layer as (inputs $\times$ units) + units (the bias term).
- **Bias** — An extra learnable constant added to each neuron's weighted sum, one per neuron, allowing the neuron to shift its activation threshold.
- **Activation function** — A non-linear function applied to a neuron's output (e.g. ReLU, softmax, sigmoid) — without it, stacking layers would collapse into one linear function.
- **ReLU** — $\max(0, x)$. Standard choice for hidden layers; cheap to compute, avoids some vanishing-gradient issues.
- **Softmax** — Turns a vector of raw scores into a probability distribution (all values in (0,1), summing to 1) — used on multi-class classification outputs.
- **Sigmoid** — Squashes a value into (0,1) — used on binary classification outputs (single unit), interpreted as a probability.
- **One-hot encoding** — Representing a category as a vector with a single 1 and the rest 0s, e.g. class "Hare" (of 3 classes) $\rightarrow [0,0,1]$.
- **Label / integer encoding** — Representing a category as a single integer, e.g. Cat=0, Dog=1, Hare=2, instead of a one-hot vector.
- **categorical_crossentropy** — Loss function for multi-class classification when y is one-hot encoded.
- **sparse_categorical_crossentropy** — Loss function for multi-class classification when y is given as plain integer labels (saves you from one-hot encoding).
- **binary_crossentropy** — Loss function for binary (two-class) classification with a single sigmoid output unit.
- **MSE / MAE** — Mean Squared / Absolute Error — standard loss and/or evaluation metrics for regression.
- **Optimizer** — The algorithm that updates the network's weights based on the gradient of the loss (e.g. 'adam', 'sgd').
- **Epoch** — One full pass of the training algorithm over the entire training dataset.
- **Batch size** — The number of samples processed together before the model's weights are updated once. Must not exceed the number of training samples.
- **Input shape** — The shape (number of features) the first layer expects per sample; must match the number of columns in X.
- **model.compile()** — Configures the model for training: sets the optimizer, loss function, and metrics to track.
- **model.fit()** — Trains the model on data for a given number of epochs and batch size.
- **model.evaluate()** — Computes loss (and metrics) on held-out data, without updating weights.
- **train_test_split** — A scikit-learn function that randomly splits data into training and test subsets (e.g. 80/20).

Final tip

In the exam, always explicitly state WHY something is wrong (the mechanism / the error Keras would throw), not just WHAT the fix is. Full marks usually require both the diagnosis and the correction.